

MODULE 3: CODING & DOCUMENTATION

Contents

1. Structured Programming.....	1
2. Object-Oriented Programming (OOP).....	2
3. Information Hiding.....	2
4. Reuse	3
5. System Documentation.....	4
6. Software testing.....	5
7. Software Testing Life Cycle (STLC).....	6
8. Types of Testing	8
9. Functional Testing	8
10. Non-Functional Testing	10
11. Testing Strategies and Levels.....	12
12. Reliability Assessment.....	14
13. Validation & Verification (V&V)	14
14. 12 Key Metrics in Software Testing	15
15. Monitoring & Control in Software Testing.....	15

Structured Programming

Structured programming is a programming paradigm aimed at improving clarity, quality, and development time. Here are the key aspects to study:

1. **Top-Down Design:** Structured programming promotes breaking down a program into smaller, simpler sub-tasks. This approach, called "top-down design," allows developers to focus on smaller components of a program individually, making it easier to manage complex systems.
2. **Control Structures:** Structured programming relies on three primary control structures to organize code logically and sequentially:
 - **Sequence:** Executes statements one after another in a fixed order.
 - **Selection:** Uses decision-making structures like if, else, and switch statements to branch the code based on conditions.
 - **Iteration:** Repeats a block of code using loops (for, while, or do-while), allowing operations to be repeated until certain conditions are met.
3. **Modularity:** Programs are organized into modules, or functions, each responsible for a specific task. This modular approach not only makes the code easier to read and maintain but also helps in identifying and isolating bugs.
4. **Readability and Maintainability:** Structured programming emphasizes writing clear and simple code. The use of modules, along with logical control structures, improves readability and maintainability. Well-organized code is easier for others to understand, test, and update.

5. **Error Reduction:** Because structured programming encourages dividing tasks into smaller functions, it can reduce errors. Debugging is simpler when issues are isolated to smaller code blocks rather than large, complex sections.
6. **Example Use Cases:** Structured programming is especially useful in procedural programming languages like C, Pascal, and early versions of BASIC, which are designed to support structured programming principles.

Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a paradigm designed to model complex systems using "objects" that represent real-world entities. Key concepts of OOP include:

1. **Classes and Objects:**
 - **Classes** are blueprints or templates for creating objects. Each class defines the properties (attributes) and behaviours (methods) that an object created from it will have.
 - **Objects** are instances of classes. Each object has its own data and can perform actions defined in its class, making it a unique entity within a program.
2. **Encapsulation:** Encapsulation is the bundling of data (attributes) and methods (functions) within a class, restricting direct access to some of the object's components. This control over data access is essential for preventing unintended interactions and maintaining data integrity. It also supports "information hiding," as users of a class don't need to understand its inner workings.
3. **Inheritance:** Inheritance allows a class (subclass or derived class) to inherit properties and methods from another class (superclass or base class). This enables code reuse, as common functionality in a superclass can be shared across multiple subclasses, reducing redundancy and improving maintainability.
4. **Polymorphism:** Polymorphism allows objects of different classes to be treated as objects of a common superclass, especially when those classes share a common interface. Two key types:
 - **Compile-time polymorphism** (method overloading): Allows multiple methods in a class to have the same name but different parameters.
 - **Runtime polymorphism** (method overriding): Allows subclasses to provide a specific implementation of a method that is already defined in its superclass.
5. **Abstraction:** Abstraction involves simplifying complex systems by modelling classes based on relevant attributes and actions. This allows a developer to focus on high-level interactions without needing to know low-level details, facilitating both design and functionality.
6. **Modularity and Reusability:** OOP supports modular design through classes and inheritance. Once a class is defined, it can be reused in other programs, reducing development time. Inheritance further enhances reusability, as it allows new classes to build upon existing ones.
7. **Example Use Cases:** OOP is widely used in languages like Java, C++, Python, and C#. It's beneficial in scenarios requiring complex data models, user interfaces, or any program where objects naturally map to real-world entities.

Information Hiding

Information hiding is a core principle in software design that focuses on restricting access to internal details of a module, class, or function to protect data integrity and prevent unintended interference

from other parts of the program. Information hiding involves encapsulating the inner workings of a module, class, or function, making only necessary information visible to the outside.

1. Why Information Hiding Matters:

- **Data Integrity:** Hiding implementation details prevents unintended data access or modifications, enhancing code reliability.
- **Reduced Complexity:** Users do not need to understand internal workings; they only interact with the inputs, outputs, and purpose of a function or class.
- **Maintainability:** Internal changes do not impact other parts of the code if the public interface stays the same, making updates and refactoring easier.

2. Examples of Information Hiding:

- **Classes and Private Variables:** In OOP, private attributes are accessed through “getter” and “setter” methods, allowing controlled access.
- **Encapsulation in Modules:** Each module hides its data and functions, exposing only essential interfaces for interaction.

3. Benefits of Information Hiding:

- **Improved Security:** Reduces the chance of accidental access to sensitive data.
- **Clear Interfaces:** Users only see what they need to interact with, which makes code easier to use and understand.
- **Easier Debugging:** Since each module is self-contained, debugging and testing become easier, as problems are likely confined to a single module rather than affecting the entire system.

Reuse

Reuse in software engineering refers to the practice of using existing code, components, or systems in new projects or parts of a project, which reduces redundancy and saves time and effort. **Types of Reuse:**

- **Code Reuse:** Reusing code modules or libraries across projects, like using sorting algorithms without rewriting them.
- **Component Reuse:** Reusing components (e.g., UI elements like buttons) or complex modules in multiple projects.
- **Design Patterns:** Reusable solutions to common design problems, such as the Singleton (single instance), Factory (object creation), and Observer (event handling) patterns.

1. Advantages of Reuse:

- **Time and Cost Efficiency:** By reusing existing code or components, developers can save significant time that would otherwise be spent on coding from scratch.
- **Improved Reliability:** Reused code is often already tested and debugged, which can lead to fewer errors in the final product.
- **Standardization:** Reuse encourages a consistent structure and reduces variability, making systems easier to understand, maintain, and extend.

2. Challenges of Reuse:

- **Integration Issues:** Code reuse can be difficult if the reused code has dependencies or conflicts with the new system.
- **Compatibility and Maintenance:** Reused code may need updates to stay compatible with future versions or new project features.

3. Best Practices for Reuse:

- **Modular Design:** Writing code in small, independent modules or classes increases the potential for reuse.
- **Documentation:** Clearly documenting reusable components helps other developers understand and integrate them more easily.
- **Use of Libraries and Frameworks:** Using libraries for common functionalities like data processing or UI components encourages reuse and reduces duplication.

4. Examples of Reuse in Software Development:

- **Standard Libraries:** Standard libraries in programming languages (e.g., Java's `java.util` package or Python's `math` module) offer reusable functionalities that developers can directly implement.
- **API and SDK Reuse:** Application Programming Interfaces (APIs) and Software Development Kits (SDKs) provide pre-built functionality, which developers can incorporate into new applications, enabling faster development.

System Documentation

System documentation is the comprehensive record of a software system's design, functionality, usage, and maintenance procedures. It serves as a vital resource for developers, users, and stakeholders, facilitating understanding, maintenance, and enhancement of the software.

1. Purpose of System Documentation:

- **Communication:** Documentation communicates essential information about the system's structure, purpose, and functionality to users and developers.
- **Maintenance:** It aids developers in maintaining and updating the system over time, even if the original developers are no longer available.
- **Training:** New team members can get up to speed quickly by reviewing the documentation.
- **Troubleshooting:** Documentation provides crucial information for debugging and resolving issues.

2. Types of System Documentation:

- **Requirements Documentation:**
 - Specifies what the system is intended to do, including functional and non-functional requirements.
 - Outlines specific features, user needs, and performance expectations to guide the development process.
- **Design Documentation:**
 - Provides details on the software's architecture and design patterns.
 - Includes diagrams such as UML (Unified Modelling Language) diagrams, flowcharts, and data models that describe the structure and interaction between components.
- **Technical Documentation:**

- Includes details on coding standards, programming languages, APIs, and third-party libraries.
- Guides developers on how to build, deploy, and troubleshoot the system.
- Covers specifics like database schemas, error handling mechanisms, and security measures.
- **User Documentation:**
 - Aimed at end-users, providing instructions on using the software.
 - Includes user manuals, help guides, FAQs, and tutorials to assist users in navigating and utilizing the system's features.
- **Maintenance Documentation:**
 - Contains information necessary for maintaining and updating the system.
 - Covers topics like system configurations, hardware requirements, troubleshooting guides, and backup procedures.
 - Important for extending functionality, updating components, and migrating systems over time.

3. Best Practices in System Documentation:

- **Clarity and Consistency:** Documentation should be easy to understand, consistent in terminology, and free of technical jargon where possible.
- **Modularity:** Organize documentation in sections based on specific aspects (e.g., setup, configuration, troubleshooting) to make it accessible and easy to navigate.
- **Regular Updates:** Documentation should evolve with the system. It is essential to update it with every major change, bug fix, or feature addition.
- **Automation and Tools:** Use tools like Javadoc, Doxygen, or Sphinx for automatic documentation generation, especially for code documentation, to ensure consistency and reduce manual work.
- **Version Control:** Document changes and maintain version histories to track the evolution of the system and provide context for future modifications.

4. Documentation Standards:

- Using standardized formats and templates helps ensure that documentation is uniform, comprehensive, and easy to follow.
- Industry standards like IEEE (Institute of Electrical and Electronics Engineers) and ISO (International Organization for Standardization) provide guidelines for documentation structure, content, and quality.

5. Benefits of Comprehensive Documentation:

- **Improves Team Efficiency:** Well-documented systems allow team members to quickly understand and work on the project.
- **Reduces Errors and Bugs:** Clear guidelines on functionality and usage reduce misunderstandings that could lead to bugs.
- **Simplifies Future Enhancements:** With detailed documentation, future upgrades and new feature implementations are easier to manage.

Software testing

Software testing is a process used to ensure that software behaves as expected and meets the desired quality standards. It involves executing software to identify defects, verify that the software functions correctly, and ensure it fulfils user requirements. **Key Aspects of Software Testing:**

1. Objectives of Testing:

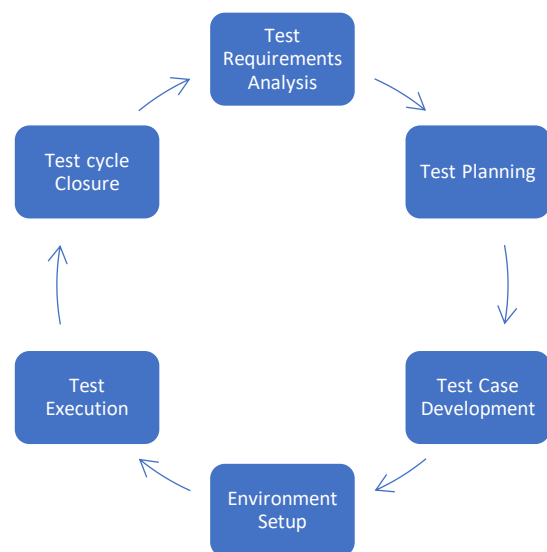
- **Find Defects:** Detect and correct bugs or issues in the software.
- **Validate Functionality:** Ensure the software works according to its specifications and requirements.
- **Ensure Quality:** Confirm that the product meets the standards of performance, security, and usability.
- **Provide Confidence:** Testing helps stakeholders have confidence that the software is working as intended.

Software Testing Life Cycle (STLC)

The **Software Testing Life Cycle (STLC)** refers to the series of phases or steps that are followed to test the software system thoroughly. It includes activities from the initial requirement analysis to the final test cycle closure. Below are the phases in the STLC:

1. Test Requirements Analysis: This is the first phase where the testing team analyses the requirements of the software to understand the scope of testing and identify the necessary resources.

- Understanding business and technical requirements.
- Identifying testable requirements.
- Reviewing documents like requirements, user stories, and acceptance criteria.
- Identifying testing objectives and deliverables.
- Defining test environments and test data.
- **Goal:** To ensure the requirements are clear, complete, and testable.



2. Test Planning: In this phase, a detailed **Test Plan** is created. The test planning phase involves deciding the testing strategy, resources, scope, and schedule.

- Defining the scope of testing (functional, non-functional, integration, etc.).
- Selecting the testing tools to be used.
- Estimating the effort and timeline for the testing.
- Assigning roles and responsibilities to the testing team.
- Identifying risks and mitigation strategies.
- Defining test deliverables (test cases, test reports, etc.).
- **Goal:** To create a comprehensive plan that guides the testing process and provides clarity on how testing will be conducted.

3. Test Case Development: Once the test plan is in place, the next phase involves **creating detailed test cases**. These test cases will be used to validate the software according to the requirements.

- Writing test cases based on the requirements and design documents.
- Preparing test data required to execute test cases.
- Reviewing test cases with stakeholders to ensure accuracy.
- Creating test scripts (for automated tests).
- **Goal:** To develop test cases and scripts that will effectively validate the software.

4. Environment Setup: This phase involves setting up the **test environment**, which includes hardware, software, network configurations, and tools required for testing.

- Preparing the required hardware and software environments.
- Setting up test databases and test data.
- Installing necessary test tools (e.g., bug tracking, automation tools).
- Ensuring the test environment mirrors the production environment as closely as possible.
- **Goal:** To ensure that the test environment is ready to execute the test cases and that all necessary tools and data are available.

5. Test Execution: During this phase, the actual **test cases are executed**, and the results are recorded. Test execution is done by running the test cases on the software, either manually or using automated test scripts.

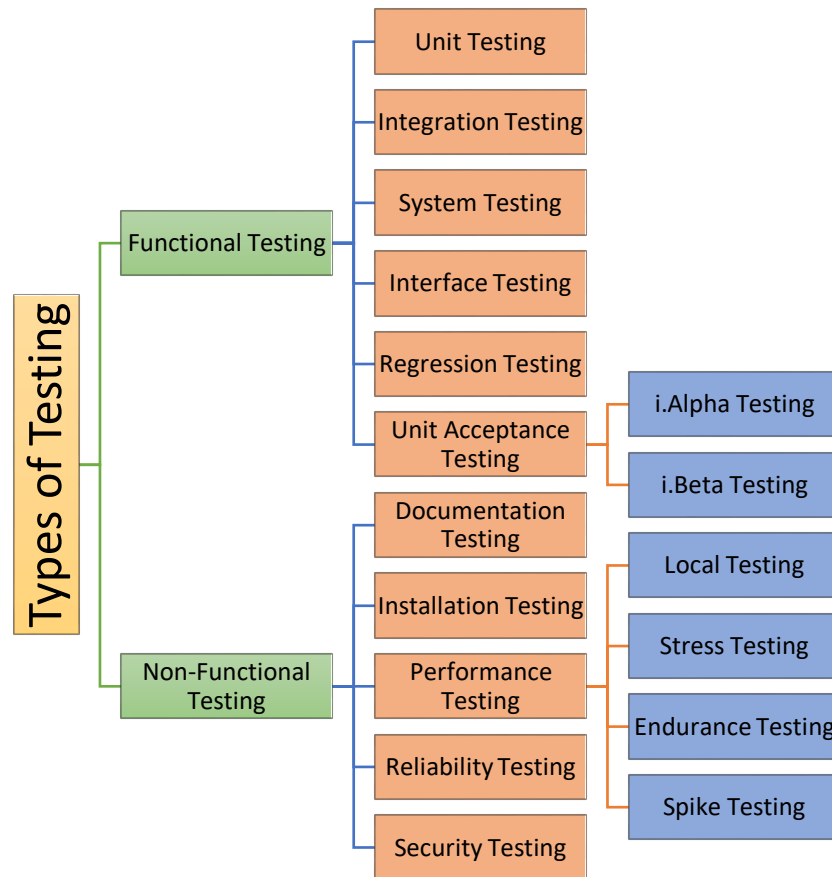
- Executing the test cases as per the test plan.
- Recording the actual results and comparing them with the expected results.
- Reporting any defects or issues discovered during testing.
- Retesting after defects are fixed.
- **Goal:** To validate whether the software works as expected and to identify any defects or issues.

6. Test Cycle Closure: Once the testing activities are complete, the **test cycle closure** phase is executed. This phase involves documenting the testing results and closing the test phase.

- Preparing test summary reports and documentation.
- Finalizing and storing test deliverables (test cases, test logs, defect reports).
- Conducting a test closure meeting with the project team.
- Reviewing the testing process to identify improvements for future projects.
- Closing open issues and ensuring that all test cases have been executed.
- **Goal:** To formally close the testing phase, ensuring that all activities are complete and that the software has been thoroughly tested.

Phase	Activities	Goal
Test Requirements Analysis	Analysing the requirements and defining test scope	Ensure requirements are testable
Test Planning	Developing test plans, schedules, and resource allocation	Plan how testing will be done
Test Case Development	Writing detailed test cases and preparing test data	Develop test scripts and data
Environment Setup	Setting up the test environment and necessary tools	Ensure environment readiness
Test Execution	Executing test cases and reporting defects	Validate functionality and identify defects
Test Cycle Closure	Finalizing reports, conducting closure meetings	Close the testing phase

Types of Testing



Functional Testing

Functional testing evaluates whether the software behaves according to the functional requirements. Types of functional testing:

- 1. Unit Testing:** Tests individual units or components of a software application to verify each part functions as intended in isolation.
 - **Scope:** Focuses on the smallest testable parts of the application, such as functions, methods, or classes.
 - **Performed By:** Typically done by developers during the development phase.
 - **Objective:** Ensures that each function or method performs as expected with different inputs, catching errors early.
 - **Tools:** Popular unit testing tools include JUnit (Java), NUnit (.NET), and PyTest (Python).
- 2. Integration Testing:** Tests interactions between integrated units or components to check if they work together as intended.
 - **Scope:** Covers combined components, such as multiple modules, APIs, or databases working in conjunction.
 - **Approach:** Common approaches include **Top-Down** (test top modules first), **Bottom-Up** (test lower modules first), and **Big Bang** (testing all components together).
 - **Objective:** Identifies interface-related defects and compatibility issues when components interact.
 - **Example:** Ensuring that data flows correctly from one module to another in a multi-layered architecture.
 - **Tools:** Tools like JUnit, TestNG, and SoapUI are used for integration testing.

3. **System Testing:** Tests the complete, integrated system to evaluate if it meets specified requirements.
 - **Scope:** Covers the entire application, including all modules, components, and interactions with external systems.
 - **Performed By:** Usually done by a dedicated testing team after integration testing is complete.
 - **Objective:** Validates the overall functionality and behaviour of the entire system under real-world conditions.
 - **Focus Areas:** Includes functionality, user interactions, performance, security, and compatibility.
 - **Tools:** System testing tools include Selenium, QTP, and TestComplete.
4. **Interface Testing:** Specifically tests the interfaces between components or systems, such as APIs, user interfaces, or third-party integrations.
 - **Scope:** Focuses on data flow and communication between systems and modules.
 - **Objective:** Verifies that interfaces send and receive accurate data and respond correctly to different inputs.
 - **Example:** Testing API calls to ensure they return correct data formats and handle errors appropriately.
 - **Considerations:** Covers areas like data validation, error handling, and compatibility.
 - **Tools:** Postman and SoapUI are commonly used for interface testing, especially for APIs.
5. **Regression Testing:** Re-runs tests after changes to the code to ensure new updates haven't broken existing functionality.
 - **Scope:** Can cover both individual modules and the entire system, depending on the impact of changes.
 - **Objective:** Confirms that bug fixes, enhancements, or new features haven't introduced new defects.
 - **Types:** Includes full regression testing (for major updates) and selective regression testing (for minor changes).
 - **Frequency:** Performed frequently, especially during continuous integration and deployment.
 - **Tools:** Automation tools like Selenium, QTP, and JUnit are used to streamline regression testing.
6. **User Acceptance Testing (UAT):** Validates that the system meets business requirements and user needs before going live.
 - **Scope:** Focuses on end-to-end workflows to ensure the software behaves as expected from a user's perspective.
 - **Performed By:** Generally conducted by business stakeholders, end-users, or a dedicated UAT team.
 - **Objective:** Ensures that the product fulfills business requirements, is user-friendly, and performs real-world tasks.
 - **Tools:** Tools like Bugzilla and JIRA help manage user feedback and defect tracking during UAT.
 - **Types of UAT:**
 - **Alpha Testing:**
 - Conducted internally by the development team and stakeholders within the organization.

- Allows early feedback and bug detection in a controlled environment.
- Often done in multiple phases to refine functionality and address issues.
- **Beta Testing:**
 - Conducted by a selected group of end-users outside the development team.
 - Exposes the software to real-world usage and gathers feedback on usability, stability, and performance.
 - Beta testers help identify final issues and areas of improvement before the full release.

Non-Functional Testing

1. **Documentation Testing:** Verifies that the software's documentation (user manuals, guides, help files) is accurate, clear, and complete.
 - **Scope:**
 - Ensures instructions for installation, usage, and troubleshooting are user-friendly and informative.
 - Checks for clarity, language accuracy, and ease of understanding for users with different expertise levels.
 - **Importance:** Clear documentation is essential for effective system use, especially for users unfamiliar with the software.
 - **Examples:** Verifying that instructions on how to install the software match the actual installation process, confirming that each feature has an adequate description, and ensuring that error messages are clearly explained.
2. **Installation Testing:** Confirms that the software installs correctly on different platforms, environments, and configurations.
 - **Scope:**
 - Ensures the software installation process completes without errors and meets system compatibility requirements.
 - Checks for dependencies, installation time, configuration options, and uninstallation procedures.
 - **Importance:** A smooth installation process improves the user's initial experience and minimizes technical support needs.
 - **Examples:** Testing installation on various operating systems, verifying that installation completes successfully without errors, and ensuring that all files are correctly placed and accessible after installation.
3. **Performance Testing:** Assesses the software's responsiveness, stability, and scalability under various conditions.
 - **Scope:** Evaluates how the system performs in terms of speed, data processing, resource consumption, and stability under different load conditions.
 - **Importance:** Ensures that the software meets performance benchmarks and can handle peak loads without significant degradation.
 - **Types of Performance Testing:**
 - **Local Testing:**
 - **Objective:** Checks how well the software performs under normal conditions on a local machine or network.
 - **Scope:** Evaluates initial load times, response times, and basic resource consumption.
 - **Examples:** Testing system start-up time and loading of essential functions.
 - **Stress Testing:**

- **Objective:** Determines how the software behaves under extreme loads, pushing beyond normal operational capacity.
 - **Scope:** Tests system robustness by increasing the load until the software fails or becomes unstable.
 - **Examples:** Simulating a sudden spike in traffic on a web application to observe when performance starts to degrade.
 - **Endurance Testing:**
 - **Objective:** Evaluates the software's stability and performance over prolonged usage.
 - **Scope:** Checks for memory leaks, slowdowns, and crashes when the software is used continuously.
 - **Examples:** Running an application for hours or days to monitor if resource usage, like memory, increases unexpectedly.
 - **Spike Testing:**
 - **Objective:** Tests how the software responds to sudden, temporary spikes in usage or load.
 - **Scope:** Observes recovery time and performance stability when the system experiences sharp increases in demand.
 - **Examples:** Simulating multiple simultaneous logins during a flash sale on an e-commerce platform to ensure smooth user experience.
4. **Reliability Testing:** Ensures that the software consistently performs its intended functions under specified conditions over time.
- **Scope:**
 - Focuses on consistency and stability, checking if the software produces expected outcomes without errors during prolonged operation.
 - Often involves testing the system under varying loads to observe how reliability might be affected.
 - **Importance:** Reliability directly impacts user trust and satisfaction, as consistent performance is essential in critical applications.
 - **Examples:** Running automated test scripts multiple times to see if they produce the same result, testing an ATM system for consistent transaction reliability over extended usage.
5. **Security Testing:** Identifies vulnerabilities and ensures that the software is protected against unauthorized access, data breaches, and other security risks.
- **Scope:**
 - Examines various aspects, including data protection, user authentication, and secure communication.
 - Involves testing for common vulnerabilities such as SQL injection, cross-site scripting (XSS), and access control.
 - **Importance:** Security is crucial for safeguarding user data and maintaining trust, especially for applications handling sensitive information.
 - **Examples:** Attempting to bypass login authentication, scanning for potential points of unauthorized access, and verifying that sensitive data, like passwords, are encrypted.

Aspect	Functional Testing	Non-Functional Testing
Objective	Validates actions and behaviours per requirements	Assesses quality and performance characteristics
Focus	Individual functions, features	System-wide performance, reliability, security
Method	Often Black-Box Testing	Black-Box, White-Box, or Gray-Box Testing
Requirements	Driven by functional requirements	Driven by quality and performance requirements
Outcome	Ensures functions work as expected	Ensures user experience, efficiency, and resilience
Tools	Selenium, QTP, JUnit	JMeter, LoadRunner, AppDynamics, Burp Suite

Testing Strategies and Levels

1. Black-box Testing: Black-box testing is a method where the tester focuses solely on the **functional requirements** of the software without any knowledge of its internal workings or code structure. The tester does not need to know how the software is designed or how it performs its tasks. Instead, the focus is on the **input-output relationships**—providing inputs and checking whether the outputs meet the expected results.

- **Focus:** Functionality, user experience, and system behaviour.
- **What the tester does:** Tests the software by providing inputs and verifying whether the outputs meet the functional requirements.
- **Internal workings are irrelevant:** The internal logic, algorithms, or code structure are not examined during black-box testing.
- **Types of black-box testing:**
 - **Functional Testing:** Validating that the software performs the intended functions.
 - **Non-functional Testing:** Checking aspects such as performance, security, and usability.
 - **Boundary Value Testing:** Checking inputs at the edge of acceptable ranges.
 - **Equivalence Partitioning:** Dividing input data into valid and invalid groups for more efficient testing.
- **Advantages:**
 - The tester doesn't need to know the software's internal code, making it easier to test from a user's perspective.
 - Useful for validating high-level functionality against requirements.
 - Focuses on the system's behaviour and ensures that it meets user expectations.
- **Disadvantages:**
 - May miss potential errors in the system's internal processes and logic.
 - Limited in covering all possible code paths and execution scenarios.
- **Example:** For a login system, a tester inputs different combinations of usernames and passwords to check if the system behaves correctly (i.e., accepting valid credentials and rejecting invalid ones).

2. White-box Testing: White-box testing, also known as **clear-box** or **structural testing**, involves testing the **internal structure** and **logic** of the software. The tester has full access to the code and architecture and tests the system's behaviour based on this detailed knowledge. This method helps ensure that the internal logic, code paths, and structures work as intended.

- **Focus:** Code structure, logic, algorithms, and paths.
- **What the tester does:** Tests individual parts of the system based on the code (such as functions, methods, or branches in the logic). The goal is to ensure that the code executes correctly in different scenarios, covering all possible code paths.
- **Internal workings are essential:** The tester needs a deep understanding of the code, architecture, and design of the software.
- **Types of white-box testing:**
 - **Unit Testing:** Testing individual units or components of the code to ensure they function correctly.
 - **Code Coverage:** Ensuring that all code paths (including if-else conditions) are tested.
 - **Path Testing:** Testing all possible execution paths in the code to ensure correctness.
 - **Loop Testing:** Checking the correctness of loops in the code.
- **Advantages:**
 - Helps identify hidden errors or issues in the internal logic and structure of the software.
 - Ensures that all code paths and branches are covered, improving test coverage.
 - Effective in detecting problems related to code, algorithms, and performance.
- **Disadvantages:**
 - Requires in-depth knowledge of the code and can be time-consuming.
 - Not suitable for testing the software from an end-user's perspective.
- **Example:** For a login system, a tester would check the function handling user authentication, inspecting whether all code paths for successful and unsuccessful logins are executed correctly.

3. Gray-box Testing: Gray-box testing is a hybrid approach that combines elements of both black-box and white-box testing. In gray-box testing, the tester has partial knowledge of the internal workings of the software but does not have complete access to the source code. This approach often uses a **limited understanding of the system's internals** to design tests while still focusing primarily on testing the software from the user's perspective.

- **Focus:** Combination of functionality and internal structure.
- **What the tester does:** The tester uses knowledge of the system's architecture, design, or database (but not the full code) to test the software. They might focus on areas such as security, session management, or application flows that involve internal systems.
- **Internal workings are partially known:** The tester might have access to high-level design or architecture documentation, but not the complete codebase. This allows them to create more targeted test cases than with black-box testing.
- **Advantages:**
 - More targeted than black-box testing because the tester has partial knowledge of the system.
 - Combines the best of both worlds by addressing both functional aspects (like black-box) and internal structure (like white-box).
 - More effective for testing system security, data handling, and integrations with other systems.
- **Disadvantages:**

- Limited by the tester's partial knowledge of the system, which may result in missing certain defects or code issues.
- Still requires careful design and planning to ensure proper coverage.
- **Example:** In the case of a web application, a tester might use knowledge of the backend database or server architecture to test for security vulnerabilities, such as SQL injection or data leakage, while still interacting with the application as a user would.

Testing Type	Focus	Internal Knowledge Required	Testing Approach	Example
Black-box Testing	Functionality, inputs, outputs	None	Tests software without knowledge of its code	Testing the login system with valid/invalid credentials
White-box Testing	Code structure, internal logic	Full knowledge	Tests the internal structure and logic	Testing the user authentication code for edge cases
Gray-box Testing	Combination of functionality and internal workings	Partial knowledge	Tests the system using partial understanding of the internals	Testing security vulnerabilities in a web application with partial backend knowledge

Reliability Assessment

- **Objective:** Measures the software's ability to function consistently without failure over time.
- **Scope:**
 - Evaluates how likely the software is to perform correctly and reliably under specified conditions.
 - Helps predict failure rates, system availability, and operational robustness.
- **Methods:**
 - **Reliability Testing:** Uses stress and endurance tests to simulate long-term usage.
 - **Failure Analysis:** Studies failure patterns to determine root causes and improve reliability.
- **Importance:** Essential for mission-critical applications, as reliability directly impacts user trust and safety.
- **Examples:** Running reliability tests on a banking system to ensure it can process transactions without error over extended usage.

Validation & Verification (V&V)

- **Objective:** Ensures that the software meets all specifications (Verification) and fulfils its intended purpose (Validation).
- **Scope:**
 - **Verification:** Confirms that the software was built correctly according to design specifications.
 - **Validation:** Confirms that the right software was built, meeting user requirements and expectations.
- **Methods:**

- **Verification Techniques:** Includes reviews, inspections, static analysis, and white-box testing.
- **Validation Techniques:** Involves dynamic testing, user acceptance testing, and black-box testing.
- **Importance:** Vital for identifying defects early in development, saving time and costs by reducing post-release issues.
- **Examples:** Verification through code reviews to ensure all modules are correctly implemented; validation by running user acceptance tests with real-world scenarios.

12 Key Metrics in Software Testing

1. **Defect Density:** Number of defects per unit of code (e.g., defects per 1,000 lines of code), indicating code quality.
2. **Defect Removal Efficiency (DRE):** Percentage of defects removed during development versus those found after release, reflecting testing effectiveness.
3. **Mean Time to Failure (MTTF):** Average time the system operates before a failure, indicating reliability.
4. **Mean Time to Repair (MTTR):** Average time required to fix a defect, showing maintainability.
5. **Mean Time Between Failures (MTBF):** Average time between successive failures, indicating stability.
6. **Test Case Effectiveness:** Ratio of passed test cases to total test cases, measuring test efficiency.
7. **Test Coverage:** Percentage of code covered by tests, showing the extent of testing.
8. **Test Execution Time:** Total time taken to execute all test cases, indicating testing efficiency.
9. **Defect Leakage:** Number of defects missed during testing but found after release, measuring testing thoroughness.
10. **Escaped Defects:** Defects detected after release, showing areas needing improvement.
11. **Test Automation Coverage:** Ratio of automated tests to total tests, indicating automation progress.
12. **Customer-Reported Defects:** Defects reported by user's post-release, a direct reflection of software quality.

Monitoring & Control in Software Testing

- **Objective:** Provides ongoing oversight of testing progress and effectiveness, allowing for adjustments to ensure quality targets are met.
- **Scope:**
 1. **Tracking Metrics:** Regularly monitor metrics (like defect density, test coverage) to identify areas for improvement.
 2. **Status Reporting:** Periodic reports on test progress, defect resolution, and test coverage help stakeholders understand testing effectiveness.
 3. **Quality Control:** Real-time monitoring identifies any deviations from quality standards, prompting corrective actions.
 4. **Risk Management:** By continuously assessing risks, teams can prioritize tests and allocate resources efficiently.
- **Methods:**
 1. **Test Management Tools:** Tools like JIRA, TestRail, or HP ALM track testing progress and defects.

2. **Dashboards:** Visual representations of metrics and progress make it easier for stakeholders to track project health.
 3. **Feedback Loops:** Regular feedback from test results and stakeholder input supports iterative improvements.
- **Importance:** Enables teams to deliver reliable, high-quality software on time and within budget by proactively addressing issues.
 - **Examples:** Using dashboards to monitor defect trends over time, providing insight into areas needing more testing or code quality improvements.